

Annotate Only What Cannot Be Inferred

A Compiler-Checked Chain from Source Document to Test Evidence, and What It Is Worth When a Machine Writes the Code

Torben Schinke

worldiety GmbH

Oldenburg, Germany

torben.schinke@worldiety.de

Preprint. Not submitted to, reviewed by, or published at any venue. Typeset in an independent, ACM-like layout; this is not an official ACM template and this work is not an ACM publication.

ABSTRACT

Machine-written code changes what a review can be: once new code exceeds what the responsible engineers can read, review by sampling becomes the working model, and the trace from a requirement to the code written for it becomes the load-bearing artefact. That trace is conventionally maintained outside the compiler, as links between identifiers that no build ever resolves. Such a link is a string, and strings rot; links recovered after the fact are heuristic and, by the recovery literature’s own account, incomplete. *speclink* takes the opposite route. It annotates only the one fact no analysis can infer — which requirement a construct exists for — in a closed whitelist grammar that is part of the ordinary build, so the host language’s compiler proves that every reference resolves. Requirements are anchored to addressable segments of the human-written source documents; drift in those segments and in the requirement wordings is caught by fingerprints; the chain is joined to recorded passing test runs rather than to markers in the source; and authorship and review are recorded from outside the code. The paper reports a design and an implementation, a structural comparison against the publicly verifiable parts of ISO 26262, IEC 61508, DO-178C and Automotive SPICE, and a delimitation from generator-side spec-driven tooling. No controlled evaluation, no conformance assessment and no behavioural verification is claimed.

CCS CONCEPTS

Software and its engineering → Software verification and validation; Requirements analysis; Automated static analysis; Documentation.

KEYWORDS

requirements traceability, static analysis, annotation compiler, functional safety, Automotive SPICE, AI-assisted software engineering, evidence, Go

1 INTRODUCTION

A software project in which a large share of the code is produced by a machine changes what a review can be. When the volume of newly written code exceeds what the responsible engineers can read, review by sampling becomes the working model rather than a lapse from it. The traditional guarantee that a change carries — that at least one person understood this code and judged it against what was asked for — no longer follows from the fact that the change was merged. It has to be established some other way, or not at all.

The evidence that this matters is not speculative. In a controlled experiment with 52 subjects, Patrick Mäder and Alexander Egyed. [39] found that subjects working with traceability were on average 21% faster and produced 60% more correct solutions than subjects without it. Across 24 open-source projects, Patrick Rempel and Patrick Mäder. [53] report that more complete traceability is associated with a decreased expected defect rate for three of the four development activities they studied. On the other side of the ledger, Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh. [50] found that participants with access to an AI assistant wrote significantly less secure code and were at the same time more likely to believe their code was secure. Delegating the judgement to another model is not an escape either: Arjun Panickssery, Samuel R. Bowman, and Shi Feng. [49] show that large language model evaluators exhibit self-preference, scoring their own outputs more highly than human annotators do, correlated with their ability to recognise those outputs as their own. A generator that certifies its own work is not a control.

Traceability from requirement to code therefore becomes the load-bearing artefact of such a project. It is also the artefact that is conventionally maintained furthest from the code: in a requirements management tool, as a graph of links between identifiers that the compiler never sees. Such a link is a string, and strings rot. Rediscovering the links after the fact has been studied for over two decades — information-retrieval recovery and analyst-in-the-loop candidate-link generation, reviewed in Section 2 — and the limits are documented by the same line of work [38]. A decade later, Orlena Gotel, Jane Cleland-Huang, Jane Huffman Hayes, Andrea Zisman, Alexander Egyed, Paul Grünbacher, and Giuliano Antoniol. [26] observe that despite advances in automated trace-link creation and maintenance, traceability implementation and use is still not pervasive in industry. The drift is not confined to trace links; the reflexion-model and documentation-consistency literature discussed in Section 2 reports the same divergence between an artefact and the code that is supposed to realise it [44, 56].

1.1 The irreducible fact

speclink starts from an observation about what a static analysis can and cannot know. Given a well-structured application, an analysis can infer a great deal: that this function type is a use case, that this struct is an event, that this type is an aggregate, that this interface is a repository, that this constant is a permission. These are structural facts, and a tool that reads the program can derive them without being told. The one fact that no analysis can ever infer is **which requirement a construct was written for**. That is a fact about intent, and it exists nowhere in the program text.

The design consequence is to shrink the annotation surface to exactly that one irreducible fact, and to make annotating anything inferable an error (README §1, rule 4). Everything the tool can derive, it derives; everything it cannot, the author states once. Because the surface is that small, it can be made compiler-checked rather than textual: a binding names a requirement by Go identifier, so the Go compiler itself proves the referent exists before speclink ever runs. A renamed or deleted requirement breaks the build instead of silently degrading into a dangling link.

1.2 speclink

speclink is an annotation compiler for Go, with a front end that reads JVM bytecode rather than source. Because Kotlin compiles to JVM bytecode, Java and Kotlin projects arrive at that front end in the same shape; one JVM profile, `java_springboot_ddd1`, exists (README §11a). It (a) recognises constructs — use cases, commands, events, aggregates, permissions, queries, projections and repositories — from the program itself; (b) requires every normative requirement to be bound to at least one construct through an annotation whose references the Go compiler proves exist; (c) anchors each requirement to an addressable segment of a human-written source document, a Markdown heading or a named region of a mockup (README §10a); (d) joins that chain to actual passing test runs by reading the test stream the build already produces (README §10b); and (e) records authorship and human review from outside the code, through `speclink attest` and `speclink freeze -reviewer` (the two are distinguished in Section 4), rather than letting the code declare it. Every finding is an error; there are no warnings and no tolerance mode, and the only escape hatch is a waiver that must carry a written reason (Section 3).

1.3 Claim versus evidence

The paper’s central conceptual contribution is a distinction that the design applies everywhere. A **claim** is something the source says: a binding stating that a construct satisfies a requirement, a marker stating that a test verifies one, a field stating that a human reviewed something. A claim is readable statically, which is what makes a **missing** claim reportable. **Evidence** is a record that something actually happened: a test that ran to the point of the marker and passed, an attestation made by a party that is not the generator. A marker in the source can sit behind a condition that never holds, or in a test that fails long before reaching it; the source cannot tell the difference. speclink therefore reports the two figures separately, and the gap between them is the number that carries information (Section 4). The same distinction is why `spec.Requirement` has no `Reviewed` field: a claim of human authorship or human review made by the author of the code is not evidence about the author.

1.4 Contributions

1. A design principle for annotation languages: annotate only what no analysis can infer — the requirement a construct was written for — and treat the annotation of any inferable fact as an error.
2. A compiler-checked binding mechanism in which requirements are ordinary program declarations and references to them are resolved by the host language’s own compiler, so a trace link cannot decay into a stale string.
3. An explicit separation of claim from evidence, applied uniformly to verification, authorship and review, together with the reporting model that follows from it: two figures, and the gap between them.
4. An anchoring of the requirement tree to segments of the human-written source documents, including named regions of mockups,

with fingerprint-based drift detection over exactly the text or pixels a requirement was extracted from.

5. An implementation covering a Go front end and a JVM bytecode front end, in which the same requirement model is populated from two different levels of program representation.
6. A discipline of measurement honesty: a direction that was not measured is reported as not measured rather than as measured clean, and the corresponding rules are not run at all.

1.5 Scope and limits

This paper reports a design and an implementation. speclink is a single-tool, single-organisation artefact, developed alongside one application framework, and the paper presents no controlled empirical evaluation: there is no comparison against a baseline, no user study, and no measurement of defect rates in projects that adopted it. The claims made here are claims about what the tool enforces and why it is built the way it is, not claims that it improves any outcome. The empirical results cited above motivate the problem; they do not validate the artefact.

The remainder of the paper first surveys traceability research, requirements management tooling and adjacent static-analysis approaches, and positions speclink among them. It then presents the requirement model and the annotation language, the construct recognition that keeps the annotation surface small, the anchoring of requirements to source documents and its drift detection, and the evidence pipeline that joins the chain to test runs. A discussion of the JVM front end shows what the design costs when the host language differs. The paper closes with the limitations of the approach and the threats to the argument.

2 BACKGROUND AND RELATED WORK

speclink sits at the intersection of four lines of work that have so far stayed apart: requirements traceability, specifications embedded in program text, architecture conformance checking, and documentation-drift detection. This section reviews each and states, for each, what speclink borrows and where it deliberately differs.

2.1 Requirements traceability

The modern framing of the problem is due to Orlena C. Z. Gotel and Anthony Finkelstein. [25], who analysed requirements traceability on the basis of an empirical study of over 100 practitioners and concluded that it is a multifaceted problem for which no all-encompassing solution exists. Their separation of pre-requirements-specification from post-requirements-specification traceability is the distinction that matters here: speclink is concerned exclusively with the post-RS direction, from a recorded requirement to the program construct written for it.

Balasubramaniam Ramesh and Matthias Jarke. [52] derived reference models for traceability empirically, from focus groups and interviews across 26 organisations, and distinguished low-end from high-end traceability users. Their work establishes that traceability practice is not uniform and that the useful unit of description is a model of what is linked to what, rather than a tool feature list.

The community consolidated this line in an edited volume [19], which contains both a chapter on traceability fundamentals [28] and the community’s grand-challenge roadmap [27]; both are cited here as pointers to that programme rather than for any specific content. The accompanying conference paper is more direct: Orlena Gotel, Jane Cleland-Huang, Jane Huffman Hayes, Andrea Zisman, Alexander Egyed, Paul Grünbacher, and Giuliano Antoniol. [26] report that, despite advances in automated trace-link creation and maintenance, traceability implementation and use is “still not pervasive in industry”.

That gap is worth closing because the benefit is measurable: the controlled experiment of Patrick Mäder and Alexander Egyed. [39] quoted in Section 1 found subjects working with traceability both faster and more often correct; a journal version of the experiment exists [40]. Traceability **quality**, not merely its existence, also matters: Patrick Rempel and Patrick Mäder. [53] studied 24 open-source projects and report, using multi-level Poisson regression, that more complete traceability is associated with a decreased expected defect rate for three of the four studied requirements-implementation supporting activities.

speclink takes these results as its motivation, and takes the “not pervasive” observation as a design constraint rather than an exhortation. Its response is that a trace which is optional will be omitted, so the trace is made a precondition of a successful build.

2.2 Recovering trace links versus declaring them

A large part of the traceability literature is concerned with **recovering** links that were never recorded. Giuliano Antoniol, Gerardo Canfora, Gerardo Casazza, Andrea De Lucia, and Ettore Merlo. [14] proposed information-retrieval based recovery — probabilistic and vector-space models — of links between source code and free text, premised on programmers using meaningful identifier names, and themselves discuss the limitations of the approach; a retrospective on that paper appeared in 2025 [15]. Jane Huffman Hayes, Alex Dekhtyar, and Senthil Karthikeyan Sundaram. [29] formulated requirements tracing as candidate-link generation with an analyst in the loop, defining goals and measures and presenting the RETRO prototype. The analyst is not incidental there: the tool proposes, a human disposes.

Andrea De Lucia, Fausto Fasano, Rocco Oliveto, and Genoveffa Tortora. [38], evaluating IR-based traceability recovery over seventeen software projects involving about 150 students, state the limitation plainly: such tools “are still far to support a complete semi-automatic recovery of all links”. That sentence is the anchor for speclink’s positioning.

speclink recovers nothing. It does not infer which requirement a construct was written for, and it does not offer candidate links for an analyst to accept. It requires the link to be declared, in the code, as a term the Go compiler already type-checks — `spec.Satisfies(...)` inside a `<base>.annotation.go` sidebar file — and it fails the build when a construct that carries business meaning has no such declaration (README §4.2); the corresponding directions are named in Section 4. The trade is explicit and unfavourable in one direction: an approach that refuses to guess cannot be pointed at a legacy system to obtain a trace matrix, which is precisely what recovery techniques are for. What it buys is that every link in the resulting record was written down by whoever wrote the code, and that a missing link is a build failure rather than a low-ranked candidate.

2.3 Specifications embedded in code

Placing specifications next to the code they describe is an old idea. Bertrand Meyer. [42] introduced Design by Contract as methodological guidelines for reliability, realised in Eiffel with assertions as the mechanism. The Java Modeling Language continues that line: Gary T. Leavens, Albert L. Baker, and Clyde Ruby. [35] embed pre-conditions, post-conditions and assertions intermixed with Java source code, written in Java expressions so that working engineers can write them; an earlier version of the notation exists [34], as does a journal overview of the associated tooling [17, 18]. For Go, the Gobra project describes itself as a prototype automated modular verifier built on the Viper infrastructure, in which specifications are written as annotations in `.gobra` programs and which has been applied to

VerifiedSCION and WireGuard [59]; the project is not a peer-reviewed source. In a different style, Daniel Jackson. [32] shows that a small declarative language for structural properties can be given a syntax “amenable to a fully automatic semantic analysis”.

speclink shares the placement and nothing else. Contracts, JML and Gobra specify **behaviour** and aim at (partial) formal verification of that behaviour. speclink specifies **provenance**: which requirement a construct exists for. Its annotations are ordinary Go terms, and what it checks is structure — that the named requirement exists, that the construct is of the kind claimed, that the binding targets a declaration the compiler has resolved, that a promised wire shape has not changed incompatibly. It does not check that the code implements the requirement, and it is not a verifier. Where behavioural evidence enters at all, it enters from outside: a passing test run is fed back through **speclink evidence**, because a test’s claim to verify a requirement is not itself evidence that it did (README §1, §2).

2.4 Architecture conformance and static analysis tooling

The reflexion-model technique [43] compares a high-level model with the source that is supposed to realise it. Gail C. Murphy, David Notkin, and Kevin J. Sullivan. [44] state the underlying observation that “the artifacts constituting a software system often drift apart over time”, and summarise consistency between a high-level artefact and the source, in an application to roughly one million lines of Microsoft Excel. A body of work follows on rule-based conformance checking as a quality-management measure [30], static conformance checking against architecture erosion [55], conformance checking for Python [36], model checking applied to conformance [41], and conformance checking for infrastructure as code [48]. These are cited only as pointers to an active area; their subject is the conformance of code to an architecture rather than to a requirement, and no claim is made here about what any of them can or cannot do.

At the practical layer, ArchUnit is a Java library that evaluates structural architecture rules over packages, classes, layers, slices and cycles as ordinary unit tests by analysing bytecode, with a .NET/C# port; it is a community project, not peer-reviewed work [16]. In the Go ecosystem, `go vet` ships with the toolchain, exits non-zero when a problem is reported and is therefore usable as a CI gate, but its own documentation describes it as heuristic guidance rather than a correctness guarantee [23]. Staticcheck adds a linter with over 150 checks aimed at bugs, performance, simplifications and style, designed for low false-positive rates and CI use, complementing `go vet` [20]. Both build on the `go/analysis` framework, whose `Analyzer/Pass/Diagnostic` model performs modular per-package analysis and whose `Fact` mechanism propagates information along the import graph, enabling “separate analysis”; the framework carries no requirement or traceability semantics of its own [24]. The Java analogue at compile time is annotation processing, in which `Processor` implementations run inside the compiler over rounds, are discovered by service-style lookup, may claim annotations and may raise errors that fail compilation [47].

speclink is an analysis of this practical kind — it recognises constructs and reports findings over a Go program, and, like annotation processing, its findings stop the build. What it adds is the subject matter: the rules are about requirements and their bindings, which none of these frameworks supplies.

2.5 Documentation drift

Nataliia Stulova, Arianna Blasi, Alessandra Gorla, and Oscar Nierstrasz. [56] observe of existing analysis tooling that “none of these tools checks for consistency of the documentation accompanying the code”, and present upDoc with an evaluation the authors them-

selves describe as preliminary. Detection approaches in this space are typically heuristic: Zhiyong Liu, Huanchao Chen, Xiangping Chen, Xiaonan Luo, and Fan Zhou. [37] report heuristic detection of outdated comments achieving 74.6% detection and 77.2% precision using 64 features.

speclink attacks the same drift, but not by inference. A requirement text and the source-document segment it was derived from are edges the compiler cannot re-derive, so they are recorded in `speclink.lock`, a file written by `speclink freeze` and never edited by hand, standing to the sources as `go.sum` stands to `go.mod` (README §6). Each recorded segment carries a fingerprint — over the text of a Markdown heading section, or over the pixels inside a declared rectangle of a mockup image. When the underlying text or image changes, the fingerprint no longer matches and the affected requirements are reported (K13-SOURCE-DRIFT); the same mechanism reports a review that outlived the declaration it covered (K18-REVIEW-STALE). The result is deterministic rather than heuristic: there is no detection rate, because the comparison is an equality test on a recorded value, and the reviewable artefact is the diff of `speclink.lock`.

2.6 Requirements management tools and interchange

Industrial practice is dominated by dedicated requirements-management platforms. IBM markets DOORS and DOORS Next with structured requirement modules, baselines, electronic signatures, multi-level traceability and a graphical link explorer, and claims support for Automotive SPICE, ISO 26262 and DO-178C [31]. Siemens claims for Polarion traceability via automatic change control, “full traceability of every source code modification up to the change request”, paragraph-level identifiability in LiveDocs, built-in ReqIF and out-of-the-box SVN and Git support [54]. Jama markets requirements traceability, standards compliance and audit trails, together with a spec-driven-development capability exposed over MCP [33]. PTC claims for codebeamer end-to-end traceability across work items in a centralised repository, OSLC-based digital-thread integration and a range of standards and tool integrations [51]. All four statements are the vendors’ own marketing claims.

Interchange between such repositories is standardised. ReqIF is an OMG specification defining an open, non-proprietary XML interchange format for requirements, motivated by cross-company supply-chain exchange [46]. OSLC is an OASIS Open Project — not an OMG standard — defining REST and Linked-Data specifications, based on RDF and the W3C Linked Data Platform, for linking lifecycle resources; its Change Management 3.0 and Configuration Management 1.0 specifications are OASIS Standards [45].

The positioning point is one of location, not capability. These platforms hold the trace in a repository beside the code and connect to it through integrations and interchange formats; speclink holds the trace in the code, as compiler-checked Go terms, and makes an incomplete trace a failing build. No claim is made here that these tools cannot enforce a trace at build time — only that no such mechanism is documented on the vendors’ product pages, and that the artefact under review differs: a repository record on one side, a source diff and a non-zero exit code on the other.

3 DESIGN

speclink is built on one rule, and everything else in this section is a consequence of it: **if a fact can be inferred from the code, annotating it is an error**. The README states this as one of the five rules that must not be broken. The emphasis belongs on **error**. A redundant annotation is not treated as harmless duplication to be tolerated or linted away at a lower severity; it is a finding, and speclink

has no severities — every finding is an error and the run either has zero findings or it fails.

The reason is that a duplicated fact is a fact that can disagree with itself. speclink recognises use cases, commands, events, aggregates, permissions, queries, projections and repositories on its own. The one thing it can never infer is **which requirement a construct was written for**. That is what an engineer annotates, and it is essentially the only thing. The annotation surface is therefore minimised not for ergonomics but for soundness: a language in which only unrecoverable facts may be stated cannot accumulate statements that drift from the program.

The rule cuts in both directions. `spec.Persistence()` and `spec.StoredAs[D]()` exist under `go_bare_ddd1`, where a hand-written interface says neither whether it is a port nor what shape reaches disk. Under `go_nago_ddd1` both terms are findings, because the repository constructors already say it — “saying it twice is the one thing this language forbids”.

3.1 The annotation language is a closed whitelist

An annotation lives in a file `<base>.annotation.go`, next to `<base>.go` in the same package, and it is part of the **normal build**. That coupling is the load-bearing decision of the whole design: a broken annotation breaks the production build, so it cannot rot unnoticed. There is no separate parser to keep in step with the language, and no way to ship a release whose traceability data no longer type-checks.

The grammar of such a file is a closed whitelist, not “valid Go”. An annotation file may contain only the package clause, imports, and package-level `var _ = spec.X(...)` terms. Table 1 gives the rejected forms and the rule code each carries.

Forbidden	Code
function definitions	SPEC-V1-001
type declarations	SPEC-V1-003
constant declarations	SPEC-V1-004
a <code>var</code> binding more or fewer than one name to one value	SPEC-V1-005
a binding not written as <code>var _ = ...</code>	SPEC-V1-006
calls into anything other than <code>github.com/worldietry/speclink/spec</code>	SPEC-V1-007
positional fields in a struct literal (use <code>Field: value</code>)	SPEC-V1-009
the address-of operator <code>&</code>	SPEC-V1-011
function literals, binary expressions, any computation	SPEC-V1-010

Table 1: The annotation subset. Everything not in the whitelist is rejected in phase V1, before the Go compiler is consulted.

The purpose of the closure is not austerity. A closed grammar turns the file into a **data language that happens to be type-checked by a general-purpose compiler**. Because no computation, no indirection and no call outside the `spec` package is admitted, every term is a literal reference to a program element, and that reference has already been resolved by `go build` before speclink runs. A requirement named in an annotation is proven to exist and to be of the right type — and speclink implements no name resolver, no import graph walker and no scoping rules of its own to obtain that guarantee. Phase V2 in the diagnostic numbering is **the Go compilation itself, reported by the Go compiler and not by speclink**: speclink’s own checking simply resumes on the far side of it.

The whitelist also closes the file-level edge. An annotation file whose neighbour `<base>.go` does not exist is an orphan and is reported as SPEC-V3-001: rename it or delete it. Without that rule

Written	Recognised	Must name a requirement?
named func type, first param <code>auth.Subject</code>	use case	yes
the same, but returning data	query	yes
type with <code>Decide(auth.Subject, *Agg) ([]Evt, error)</code>	command	yes
type with <code>Evolve(ctx, *Agg) error and Discriminator()</code>	event	yes
type with <code>Identity()</code> ID, or the type an event's <code>Evolve</code> folds into	aggregate	no
<code>permission.Declare...[UseCase](id, ...)</code>	permission, bound to that use case	no
state type of <code>evs.NewProjection / evs.NewSingleton</code>	projection	yes
type <code>X data.Repository[E, ID] or = data.ReadRepository[...]</code>	repository	no
<code>hapi.Post[In](api, hapi.Operation{Path: ...})</code> and the chain on it	endpoint, with its wire types	no

Table 2: What speclink infers, and from what.

the sidecar convention would be an aspiration rather than a fact, and the per-file impact analysis that depends on it would silently answer for a file that is no longer there.

3.2 Bindings and assertions

The language has four bindings, and they are its entire side-effecting surface. `spec.For[T](...)` attaches to a named type: a use case func type, an event, an aggregate, a projection state. `spec.ForDecl(ref, ...)` attaches to a declared function, variable or constant, **named by the value itself**, so the Go compiler proves it exists. `spec.ForField[T]("Name", ...)` attaches to a struct field of T. `spec.ForPackage(...)` attaches to the package of the neighbouring file.

Everything else is an assertion: a pure term passed into a binding, never standalone — `spec.Satisfies`, `spec.Transition`, `spec.External`, `spec.Help`, `spec.Term`, `spec.Rationale`, `spec.Waive`, `spec.Draft`, `spec.Optional` and `spec.StoredAs`. Of these, `spec.Satisfies(reqs...)` binds the construct to one or more requirements and is “the central statement, and the only one that cannot be inferred”.

```
package sales

import (
    "example.com/erp/requirements/fun/quote"
    "github.com/worldiety/speclink/spec"
)

var _ = spec.For[SubmitQuote](
    spec.Satisfies(quote.RQuoteSubmit),
    spec.Help(`Submit the approved quote.`),
)
```

`ForField` is the one deliberate exception, and it is worth stating plainly rather than hiding. It takes the field name as a string, the only reference in the whole language that the Go compiler cannot check: Go has no expression denoting a struct field independently of a value, so the guarantee cannot be obtained the way `ForDecl` obtains it. speclink therefore checks the name itself against the type. This is the residue of the compiler-delegation strategy, minimised to exactly one construct rather than pretended away. A field binding also satisfies a requirement **for that field**, not for the type it belongs to.

3.3 Requirements as compiled values

A requirement is a `spec.Requirement` value in a file `<ID>.spec.go`, one requirement per file, the file named after the ID. Requirements may only appear in the requirement tree, never in an annotation file: a requirement is owned by the domain side and outlives any implementation.

Three fields carry the semantics the coverage rules read. `Kind` is `Functional`, `NonFunctional`, `Constraint` or `Decision`, and a `Decision` must carry a `Rationale`. `Discipline` is `Business`, `Technical` or `Mixed`. `Status` is `Normative`, `Abstract`, `Planned`, `OutOfScope`, `Informative` or `Superseded`, and it decides what coverage means: only `Normative` must be covered; `Abstract` is a pure derivation node and must **not** be covered directly (K3-ABSTRACT-COVERED); `Superseded` must no longer be covered (K3-SUPERSEDED-COVERED). A normative requirement satisfied by nothing is K3-REQ-UNCOVERED. Coverage is thus a status-indexed predicate rather than a uniform one, which is what lets a tree be grown, refactored and retired without either a false gap or a false claim appearing in the figures.

`DerivedFrom` and `Supersedes` reference other requirements by their **Go identifier**, never by the ID string. Moving a file, or reorganising a domain directory, therefore cannot break a reference — the compiler re-resolves it — while a reference to a deleted requirement fails the build rather than dangling. Cycles in `DerivedFrom` are reported (SPEC-V5-013), because acyclicity of a value graph is the one property the compiler will not check.

Layout is where the design deliberately does the opposite of minimising redundancy. `Directory`, `ID` prefix and `Kind` state the same fact three times, and speclink checks that all three agree: SPEC-V5-032 when the prefix contradicts `Kind`, SPEC-V5-033 when the directory does, SPEC-V5-035 when the domain directory contradicts the `ID` prefix. This is not a violation of the inference rule but its mirror image. The three statements are not one fact annotated three times; they are three independent encodings maintained by three different mechanisms — a file system, a naming convention and a typed constant — used as a checksum on each other. Redundancy that can be mechanically compared is a detector; redundancy that cannot is drift.

3.4 What is inferred

Table 2 gives the Go frontend’s recognisers under `go_nago_ddd1`. The left column is what an engineer writes anyway; the middle column is what speclink derives from it by resolved type, so an alias import or a renamed identifier can neither fool it nor help.

The third column is where the inference rule pays out. Aggregates, permissions and repositories are structural: they are covered through the use case that guards, writes or holds them, so nothing is written about them at all. Everything else names its own requirement — one `spec.Satisfies` term, and no second fact. A query is no exception, because reading is a promise too; a projection is none either, because it crosses aggregates and answers a question no single use case states.

Endpoints are read from the builder rather than declared: the method from the name of the call, the path from the `Operation`

literal, and the request and response types from the type arguments the compiler has already resolved. The whole chain is followed, and the use case behind it is found by type, so no wrapper or middleware has to be recognised for the route to be seen. speclink never guesses a wire shape — on a bare mux the columns stay empty and the catalogue says `_not stated here_`.

3.5 Processes and topology

A process is declared in a `<name>.process.go` file as a `spec.Process` value with **Nodes** and **Edges**: the course of business — not what there is, but what happens, in which order, and where it can end — and it satisfies requirements the way a construct does.

The decision that matters is that **a step names a construct, not a caption**. `spec.Do[sales.SubmitQuote]("abgeben")` is a type argument, so the Go compiler establishes that the named thing exists — and then **K16-NODE-REF-UNKNOWN** establishes that it is the right **kind** of thing, refusing an activity that names an event and an event that names a use case. This is the two-stage pattern of the whole tool in miniature: existence from the compiler, meaning from speclink. A step written as prose would be a caption, and a caption cannot go stale in a way anybody notices.

The vocabulary is nine nodes — **Start**, **End**, **Do**, **Emit**, **On**, **Fork**, **Join**, **Choice**, **Merge** — and the omissions are deliberate: inclusive and event-based gateways, boundary events, timers, compensation and subprocesses are absent until a real case asks for one. Because a process is a graph rather than nested blocks (real courses come back), the compiler cannot check the wiring; that price is paid by **K16**, which refuses a dangling edge, a duplicated name, a node no start reaches and a node from which no end can be reached. What is **not** established is stated as such: speclink does not attempt to decide whether every fork is matched by exactly one join on every path, and the rule set says deadlock freedom is not established rather than implying otherwise.

A process lives above the bounded contexts, in a package of its own, for a structural reason: it is precisely the thing no single context owns, and a context that declared it would have to import its neighbour, which the layering rules forbid.

Topology files, `<name>.topology.go`, declare `spec.Actor`, `spec.Foreign` and `spec.Channel` values. This is the one part of the model that is declared rather than read, and not for convenience: no Go module states that an end user exists, that the object store is somebody else's responsibility, or that the channel to it carries customer data under a short-lived key. That is not missing from the code; it is knowledge the code cannot hold — the inference rule applied honestly in the direction where inference is impossible. **Protocol**, **Data**, **Auth** and **Crypto** are required fields, and what keeps the result from being a picture is that both ends are enumerated: a channel naming a package that is not there is a mistake, and an adapter no channel names is a way out that never appeared in any interface list.

3.6 Promised shapes and persisted data

An event struct is the wire format of a log that is replayed forever, so every field name and every field type is a promise from the moment the first message is stored. Two things are persisted and only two: an event, whose struct is the stored form because it implements **Evolve** and carries a discriminator; and a persistence model, the type a repository was built over.

Everything persisted is frozen by default, and `spec.Draft()` is the exception. The inversion is deliberate: the number of drafts in a system shrinks over its lifetime, so marking the exception costs less than marking the rule, and forgetting fails safe — an unmarked newcomer is frozen at once, which surfaces as an error the first time somebody changes it rather than as unreadable data a year

later. `spec.Draft()` claims one specific thing, and it is not about deployment: that every stored message of this type may be deleted. The term cascades over package, type and field, and marking a level the cascade already covers states nothing new and is reported as **K9-DRAFT-REDUNDANT** in phase V4 — the inference rule again, applied to speclink's own terms.

Once a shape is frozen: the discriminator must never change and must be unique in the module; a field must not be removed, nor renamed on the wire (the Go field name may change freely, the JSON tag is what is promised), nor change its shape incompatibly. New fields are allowed but must be marked `spec.Optional()`, and optionality cannot be withdrawn. Two changes are free, because the record cannot tell them apart from no change: a named type over the same underlying type, and any change of integer width, since a promise about an integer field is that it holds a whole number.

None of this is decidable from the current source — nothing in a working tree says what a field used to be — so the promise is recorded in `speclink.lock`, written by `speclink freeze` and never edited by hand. It is not a second place to state intent; intent stays in the code, where the field type says the shape and `spec.Draft` says the status. The lock records what has already been committed to, the same relation `go.sum` has to `go.mod`, and `freeze` refuses to record anything that already breaks a promise.

3.7 Escape hatch and diagnostics

The only escape hatch is `spec.Waive(rule, reason)`, and the reason is mandatory. A waiver leaves a trace in the report and counts toward the figures, so suppression is visible rather than silent. What a profile deliberately cannot do is switch a rule off; that is `spec.Waive` with a reason, or the scope, and a third way would be severities under another name. Scope narrows package by package rather than rule by rule, on the ground that **this package is not under speclink yet** is a true statement while **this rule half applies here** is not.

Diagnostics are prescriptive. Each finding states what is wrong, why it is wrong, and what to do, with the **How**: line usually the literal fix:

```
file.go:12:6: [SPEC-V6-056] use case
SubmitQuote has no permission of its own.
A permission per use case is what makes
authorisation assignable and auditable. ...
Add `PermSubmitQuote = permission.Declare
[SubmitQuote](".", name, description)` and
check it in NewSubmitQuote.
```

The code is **SPEC-<phase>-<number>**, and phases run in order, each only when the previous one is clean: V1 the annotation subset, V2 the Go compilation, V3 a binding on an illegal target, V4 an annotation stating a fact already established elsewhere, V5 the requirement tree, V6 the specification and architecture rules. The ordering is itself a design element: a project is not told about a coverage gap while its annotation file is not yet a legal member of the language.

3.8 Language independence

speclink separates a language-independent core from a per-language front end; the Go reader lives under `internal/lang/golang` and the JVM reader under `internal/lang/jvm`. The rule families split along the same seam. The language-independent families live in `internal/check` — **K1**, **K3**, **K9**, **K10** and **K12** to **K18**, and **K20** — and the requirement-tree families **K11** and **K19** live in `internal/reqtree`. Only **K4** to **K8** belong to a profile's style, and those are implemented in `internal/lang/golang`. The seam is a property of the tree rather than of a stated interface; Section 7 records where each family sits.

The JVM front end reads **compiled classes** rather than source, on the same delegation argument as the Go whitelist: a class file has already been resolved, so every type it names is fully qualified, every supertype is named outright and every annotation has had its import worked out. It also collapses three targets into one — Java on Spring, Kotlin on Spring and Kotlin on Android — since Kotlin compiles to JVM bytecode and is only dexed afterwards (README §11a).

Because a Java annotation may hold only compile-time constants, a requirement is a class, and one requirement names another by class literal (derivedFrom = RDecNumbering.class); a construct binds with @Satisfies(RQuoteSubmit.class). With a string there the reference would be unverified, “which is the one thing this design will not accept”. There is no library to depend on: a project declares these annotations itself and speclink recognises them by fully qualified name, the same way it recognises a framework without linking it. Verification is read from @Test @Verifies(...) in the bytecode joined against Surefire’s XML on the name a report gives a test.

The honest part is what the front end refuses to answer. The java_springboot_ddd1 profile prescribes no architecture rules and has no persistence recogniser, and a run under it prints not measured: schema evolution, because this frontend reads no persisted shapes beside its summary rather than reporting the family sound. A direction that was never measured must not read as one that came out clean — over an empty set the evolution rules would report every recorded type as removed, which is a different claim from “nobody looked”. The figure is absent rather than zero for the same reason.

4 FROM CLAIM TO EVIDENCE

A marker in the source is a **claim**. A line written while the program ran is **evidence**. The two are not degrees of the same thing: a claim is a statement that somebody wrote something down, and a static analysis can read it, count it and report its absence, but it cannot report its emptiness. A binding can name a requirement that the code does not honour. A verification marker can sit behind a condition that never holds, or in a test that fails long before reaching it, and the source cannot tell that apart from a working test by any amount of analysis (cmd/speclink/evidence.go). Evidence is the opposite trade: it can only be produced by execution, it can never be re-derived from the working tree, and for exactly that reason it has to be written down.

A green run is a statement about **four directions**, and the term is used in that sense throughout this paper. **speclink verify** prints one figure for each and all four must reach 100% (README §2): **accounted**, that every segment of every source document became a requirement; **bound**, that every construct carrying business meaning names a requirement; **covered**, that every normative requirement is named by at least one construct; and **verified**, that something in the source claims to demonstrate it. A fifth figure, **demonstrated**, reports how many of those claims were observed to run; it is a figure rather than a threshold, and the finding that accompanies it is K14-VERIFICATION-STALE.

The chain those directions run along has four links, held up by three different kinds of check.

Source segment → requirement the step with no formal semantics. The anchor is resolved mechanically — a missing heading slug or region name is a finding — but whether the requirement says what the segment means is not decidable, and speclink does not pretend otherwise. What is mechanised is that the segment was read at the wording it then had: fingerprint-checked, K13-SOURCE-DRIFT.

Requirement text → its own past fingerprint-checked against speclink.lock, K10-REQ-CHANGED.

Requirement → construct compiler-checked. A binding names a requirement by Go identifier, so the referent is proved to exist by the Go compiler before speclink runs.

Requirement → passing test run-time evidence, recorded by speclink evidence from the test stream the build already produces. The claim side of the same link — that a marker exists at all — is static, and its absence is K14-REQ-UNVERIFIED.

Provenance and human review sit beside the chain rather than inside it, and are recorded from outside the code entirely.

link	held up by	rule
segment → requirement	fingerprint	K13-SOURCE-DRIFT
requirement wording	fingerprint	K10-REQ-CHANGED
requirement → construct	Go compiler	—
claim of a test	static read	K14-REQ-UNVERIFIED
passing test run	run-time record	K14-VERIFICATION-STALE
human review	fingerprint	K18-REVIEW-STALE

Table 3: The chain from a source segment to a recorded test run, and what holds each link up.

4.1 The step with no formal semantics

Above the requirement tree are the documents people actually wrote. speclink recognises two kinds of source and no others (README §10a). A Markdown file is segmented by its headings, and the anchor is the heading slug. An image — .png or .jpg — is segmented by a sidcar manifest <image>.speclink.json that declares named regions as pixel rectangles with origin at the top left (internal/source/image.go). From the requirement side there was never a difference between pointing at a section and pointing at part of a screen; both are anchors into a sequence of segments.

PDF is deliberately unsupported. The instruction is to convert it to Markdown, which makes the conversion a visible step and the result diffable in the pull request — as opposed to a conversion hidden inside the tool, where nobody reviews it.

Regions are declared, not detected. An image is not decomposable by any deterministic rule, and a model inventing regions would only move the invented-requirement problem one level down: instead of requirements with no source, there would be sources with no author.

Every segment must yield at least one requirement. Some genuinely carry no obligation — a title, an introduction, a glossary, a chrome element — and that is stated where the segment is written: the line <!-- speclink:informative --> in the Markdown (internal/source/markdown.go, InformativeMarker), or "informative": true on the region in the manifest.

This is not spec.Waive and cannot be. A waiver attaches to a Go construct, and a document section has none, so a waiver narrowed to one section could not be written down at all. Putting the statement in the document also puts the decision with the person who wrote the section, and keeps the fact in one place rather than splitting it between a paragraph and a package.

This step should be read as the honest weak point of the design. It is the one link in the whole chain that a compiler cannot hold up. Everything below the requirement tree — the binding, the construct recognition, the coverage figure — is only as meaningful as the extraction that produced the tree, and no rule in speclink can establish that a requirement faithfully renders the paragraph it was taken from. What the tool can do is make the pairing readable: a single output

places each requirement next to the segment it came from, so a person can judge the extraction, and record that a person did.

4.2 Drift as a fingerprint, not a heuristic

A requirement text and a source segment are the same kind of edge as a persisted wire format: the compiler cannot re-derive them, so `speclink.lock` records what they used to say (`internal/baseline`, `internal/check/drift.go`).

Rewrite the text of a requirement and its identifier is unchanged, every `spec.Satisfies` still compiles, and the coverage figure stays at 100%. `K10-REQ-CHANGED` is the only thing that notices. Rewrite the section it was extracted from and the anchor still resolves; `K13-SOURCE-DRIFT` is the only thing that notices.

For images the fingerprint covers the pixels inside the declared rectangle, and the coordinates are what make the report specific. Recolouring one button reports the requirements of that button and nothing else. A layout shift moves everything below it and reports drift across the board — coarse, but visible, and resolved by one freeze, which is preferable to a report so blunt that it gets ignored (`internal/source/image.go`).

Reformatting is not drift and reflowing is not a rewrite; both cases are pinned by tests, because a rule that fires on a formatter is a rule that gets waived out of habit and then stays waived.

Both findings are answered the same way: re-read what the finding names, change what has to change, then run `speclink freeze`. The diff of `speclink.lock` is the review. That is the reason for a recorded fingerprint rather than a status field somebody maintains: a moment of review that appears as a diff in a pull request survives contact with a project, where a discipline does not (`cmd/speclink/freeze.go`).

4.3 Verification: the two lives of one call

At the end of a test, the test says what it demonstrated:

```
func TestSubmitDrawsAGaplessNumber(t *testing.T) {
    // ... exercise the use case, assert ...
    spec.Verified(t, quote.RQuoteSubmit)
}
```

Position matters here and nowhere else in the language. `spec.Verified` writes a line when it runs, so putting it at the end says the test got there; putting it at the top says only that the test started.

The call has two lives. It is read statically, which is what makes a **missing** call reportable as `K14-REQ-UNVERIFIED`. It writes a line when it executes, which is what makes a **present** call believable.

in the source	recorded	meaning
no call	–	nothing claims to verify it — <code>K14-REQ-UNVERIFIED</code>
a call	nothing, or against an older wording	claimed, not shown — <code>K14-VERIFICATION-STALE</code>
a call	matching, from a passing test	demonstrated

Table 4: The three states of one requirement under verification (README §10b).

The second half runs as the fourth step of the build order:

```
go test -json ./... | speclink evidence
```

Only passing tests are recorded: a test that claimed something and then failed showed nothing, and recording it would make the failure invisible. A run is the whole truth — a requirement that nothing demonstrated this time loses its record. `K14-VERIFICATION-STALE`

then reports it, and the same finding covers three distinct mistakes that are indistinguishable from the source and are all fixed the same way: the call sits behind a condition that never holds; the test fails before reaching the end; or the requirement was rewritten after the last run, so the evidence that exists was evidence for other words.

The summary therefore reports a pair, **100% verified, 88% demonstrated**, and the gap is the interesting number. It says that a test exists, compiles and claims something, and has not been seen doing it. A single figure would have to choose between over- and under-reporting; the pair reports the uncertainty instead of resolving it by fiat.

`speclink` deliberately does not run the tests itself. The build order is compiler, then `speclink`, then tests, and a command that invoked the suite would either violate that order or duplicate it. It also makes the evidence something CI hands over rather than something `speclink` produces, which is the right way round for evidence: the party that asserts a result should not also be the party that grades it.

Some requirements cannot be tested at all. A structural decision — **customer data is stored as state, not as facts** — is discharged by the type existing, and a test could only assert that the code compiles as written. Such a requirement is waived on a construct that satisfies it:

```
var _ = spec.For[Customer](
    spec.Satisfies(dec.RDecCustomerState),
    spec.Waive("K14-REQ-UNVERIFIED", "..."),
)
```

The waiver attaches to a construct, not to a requirement, because `spec.Waive` attaches to a Go construct and a requirement declaration is not one. The finding names a construct that the waiver can be put on, so the escape hatch is always anchored somewhere the compiler can see it, and always carries a written reason.

4.4 Provenance and review, recorded from outside

Authorship and human review are recorded by commands, never declared in the code (README §2, §10a, `cmd/speclink/attest.go`, `cmd/speclink/freeze.go`). Two commands carry a `-reviewer` flag and they record two different facts. `speclink attest -reviewer` names one or more **declarations** — a construct, by name or by package pattern — and records that the named person read them; `speclink freeze -reviewer` records that the named person read the **requirement wordings** as they then stood, bound to those wordings, so that rewriting a requirement text discards the review. Every later mention in this paper follows that split.

```
speclink attest -origin llm ./app/sales/...
speclink attest -reviewer "TS" SubmitQuote
speclink freeze -reviewer "Frau Meier" ./...
```

There is no Reviewed field on `spec.Requirement` and no authorship field in the code. The reason is the claim/evidence distinction applied to the author: the same machine that writes the code writes the annotation, so a self-declared claim of human authorship or human review is not evidence about the author. Routing the certification to another model is not an escape either, for the self-preference reason given in Section 1 [49]. The human end of the same failure is documented too: Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh. [50] found that participants with access to an AI assistant wrote significantly less secure code and were at the same time more likely to believe it secure, which is the human end of the same failure: the confidence attached to generated code is not calibrated to it.

Reviews are targeted rather than run-wide. A reviewer is usually specialised and reads a few declarations at a time; recording a whole run as read because somebody looked at one use case is the fastest

way to make the figure meaningless. A use case’s fingerprint therefore covers its constructor and not only the named func type: the signature is a line, the logic a reviewer actually reads lives in the **New...** beside it, and a fingerprint over the signature alone would survive any rewrite of the body and go on claiming the review still holds. When the subject does change under a recorded review, K18-REVIEW-STALE reports it — somebody’s name attached to text they never saw, which nothing else in the record can detect.

There is deliberately no finding for unread code. In a project whose code a machine writes and a person samples, a build that stays red until everything has been read is never green, and a signal that is always on is not a signal. It is reported as a figure instead:

```
8 declarations (8 machine written, 2 read by a person)
9 requirements (8 normative, 3 reviewed)
```

Unattested is neither: a declaration nothing has said anything about counts as neither machine written nor read, because silence must not be able to pass for handwork. And speclink writes down what it is told and checks none of it. If whatever drives the generator may also invoke -reviewer, the record is worth nothing; what keeps it honest is who is permitted to make which call, which is an organisational control and not speclink’s to decide. Saying so plainly is better than a mechanism that implies a guarantee it has not got.

5 RELATION TO NORMATIVE AND PROCESS-ASSESSMENT PRACTICE

The normative text of most of the documents discussed here is not publicly available. ISO 26262, IEC 61508 and IEC 62304 are sold per part; DO-178C, DO-330, DO-333 and their EUROCAE counterparts are sold by RTCA and EUROCAE. What is publicly verifiable is a narrow set: part titles and scopes, the clause **titles** that the publishers expose in free previews, the definitions published in the ISO Online Browsing Platform, and the descriptive text the publishers themselves write on their catalogue pages. The one exception is Automotive SPICE: the VDA QMC publishes the complete Process Reference Model and Process Assessment Model as a free PDF [12], and that document was read directly.

This section therefore compares speclink against what can be checked, and nothing else. No clause numbers beyond those visible in a public table of contents, no objective numbers, no integrity-level or tool-qualification-level scheme, and no requirement text are stated anywhere below, because none of them could be verified from a primary source. More importantly: **no claim of conformance is made**. speclink has not been assessed, qualified, certified or audited against any of these documents. What follows is a structural comparison — an argument about where a compiler-checked link sits relative to what these regimes are organised around — not a conformance argument, and not a tool-qualification argument.

5.1 Functional-safety standards

ISO 26262 is positioned by its own text as a sector adaptation: “The ISO 26262 series of standards is the adaptation of IEC 61508 series of standards to address the sector specific needs of electrical and/or electronic (E/E) systems within road vehicles” [9]. That relationship is not incidental to the present discussion; IEC 61508-1 states as a major objective “to facilitate the development of product and application sector international standards by the technical committees responsible for the product or application sector” [1], and ISO 26262 is precisely such a product. IEC 61508-1 also carries the status of a basic safety publication according to IEC Guide 104 [1].

ISO publishes ten parts of the 2018 edition as a package, from Part 1 “Vocabulary” to Part 10 “Guidelines on ISO 26262”; two further parts,

on semiconductors and on motorcycles, are cited in the bibliography of Part 8 [9]. Two parts matter here. Part 6, “Product development at the software level”, specifies requirements for product development at the software level, covering — in the wording of its own scope — the specification of software safety requirements, software architectural design, software unit design and implementation, software unit verification, software integration and verification, and testing of the embedded software; it also addresses configurable software [10]. Part 8, “Supporting processes”, lists twelve topics in its scope, among them overall management of safety requirements, configuration management, change management, verification, documentation management and — the entry that concerns any tool author — **confidence in the use of software tools** [9]. The series is based on a V-model as reference process model and cross-references clauses in an “m-n” notation, where “m” is the part and “n” the clause within it [9].

Two published definitions are worth quoting because they are load-bearing for the rest of this section. ASIL is defined as “one of four levels to specify the item’s or element’s necessary ISO 26262 requirements and safety measures to apply for avoiding an unreasonable risk, with D representing the most stringent and A the least stringent level”, with the note that “QM is not an ASIL” [11]. And a **software tool** is “computer program used in the development of an item or element” [11] — a definition under which speclink plainly falls.

A verifiable and instructive fact is that these bodies of practice cite one another. The bibliography of ISO 26262-8:2018 lists, among others, RTCA DO-178C, CMMI for Development, Automotive SPICE (marked as “an example of a suitable product available commercially”), ISO/IEC/IEEE 12207, ISO/IEC/IEEE 29148 and the ISO/IEC 33000 series [9]. The domains are not sealed off from each other; a tool argument made in one vocabulary is at least legible in the others.

IEC 61508 itself consists of Parts 1 to 7, Edition 2.0, dated 2010-04-30, under IEC TC 65/SC 65A [1]. Part 3, “Software requirements”, is explicit about tooling: it “provides specific requirements applicable to support tools used to develop and configure a safety-related system” and, together with Parts 1 and 2, “requirements for support tools such as development and design tools, language translators, testing and debugging tools, configuration management tools” [2].

In civil aviation, RTCA states that “Compliance with the objectives of DO-178C is the primary means of obtaining approval of software used in civil aviation products” [3]. Tool qualification is handled by a separate document: RTCA’s description of DO-330 says it “explains the process and objectives for qualifying tools”, where a tool is “a computer program or a functional part thereof, used to help develop, transform, test, analyze, produce or modify another program, its data or its documentation”, and notes that it “may also be used by other domains, such as automotive, space, systems, electronic hardware, aeronautical databases and safety assessment processes” [4]. DO-333 supplements DO-178C and DO-278A for formal methods [5]. EUROCAE’s own training material describes ED-12C as “the European reference standard for airborne software certification” and “equivalent to RTCA DO-178C” [21]. For medical devices, IEC 62304, in its consolidated Edition 1.1 combining the 2006 first edition with its 2015 amendment, “Defines the life cycle requirements for medical device software” and applies where software is itself a medical device or an embedded part of one, while explicitly not covering validation and final release of the device [6].

The speclink-facing observation is structural. What these regimes place at the centre — management of safety requirements, configuration management, change management, verification, documentation management, and confidence in the use of software tools [2, 4, 9] —

ASPICE element [12]	speclink mechanism	held up by
SWE.1.BP5 / outcomes 5–6: consistency and bidirectional traceability for software requirements	Sources on a spec.Requirement; DerivedFrom between requirements	K11-REQ-UNSOURCED and K11-SOURCE-UNANCHORED, then K13-SOURCE-DRIFT and K10-REQ-CHANGED
SWE.3 outcome 3: traceability between source code and detailed design	spec.For / ForDecl / ForField / ForPackage with spec.Satisfies	Go compiler, then K1-CONSTRUCT-UNBOUND and K3-REQ-UNCOVERED
SWE.4 / SWE.6: traceability to verification measures and results	spec.Verified(t, req) recorded by speclink evidence from the test stream	a recorded passing test run
Note 11: links alone do not mean the information is consistent	fingerprints over requirement text and source segments	speclink.lock, re-freeze on drift
work product “13-51 Consistency Evidence”	the generated specification document and the summary figures	speclink generate, speclink verify

Table 5: Correspondences between Automotive SPICE elements and speclink mechanisms. The right-hand column names what actually holds the link up; a correspondence is not a conformance claim.

is exactly the set of concerns that a tool producing evidence **about** a safety-related product inherits. If the tool’s output is to carry weight, the tool is itself an object of scrutiny.

speclink’s design has affinities with that posture, and they are deliberate. Every finding is an error: there are no warnings, no severities and no tolerance mode, so a run either has zero findings or fails (README §1). The single waiver mechanism described in Section 3 leaves a trace in the generated report — which is also how the statement of applicability for an external standards catalogue falls out, since “applicable”: false without a “because” is refused (README §4.5). Most references are not resolved by speclink at all: `DerivedFrom`, `Supersedes` and `spec.Satisfies` name Go identifiers, so the Go compiler proves the referent exists before speclink runs, and `spec.Do[T]` in a process names a construct rather than a caption (README §4.1, §4.3). And the final link in the chain is a recorded passing test run rather than a claim in the source: `speclink evidence` reads the test stream, because “a claim that a test verifies something is not evidence that it did” (README §1).

This is a design affinity and nothing more. It is **not** tool qualification. speclink has undergone no qualification activity of any kind, no qualification plan exists for it, and none of the publicly available material cited above would be sufficient to construct one. The most that can be said is that the tool was built so that its own output is auditable in the same register the standards use.

5.2 Automotive SPICE

Because the PAM is free, this is the one comparison that can be made against primary normative text. Automotive SPICE is published by the VDA QMC; the current version is the Process Reference Model / Process Assessment Model Version 4.0, authored by VDA Working Group 13, dated 2023-11-29 and marked “Released” [12, 58]. The model states that it “was developed in accordance with the requirements of ISO/IEC 33004:2015” and that its measurement framework is “an adaption of ISO/IEC 33020:2019”, with an ISO/IEC 33003:2015 compliant measurement framework defined in its section 5 [12]. The lineage back to the 15504 series is acknowledged in the model’s own words — “a PRM/PAM according to ISO/IEC 33004 (formerly ISO/IEC 15504-2)” [12] — and independently by ISO, whose catalogue page for ISO/IEC 33002:2015 lists ISO/IEC 15504-2:2003 as its withdrawn predecessor [7]. The capability dimension has six levels, 0 to 5, incorporating nine process attributes from PA 1.1 Process performance to PA 5.2 Process innovation implementation; level 0 is defined as “The process is not implemented or fails to achieve its process purpose” [12]. The process dimension comprises 32 processes in 3 categories and 11 groups [58].

The software engineering group of version 4.0 contains six processes, named in the PAM as SWE.1 Software Requirements Analysis, SWE.2 Software Architectural Design, SWE.3 Software Detailed Design and Unit Construction, SWE.4 Software Unit Verification, SWE.5 Software Component Verification and Integration Verification, and SWE.6 Software Verification [12]. Version 4.0 renamed SWE.3 and SWE.5 relative to earlier versions, and Annex C of the PAM still refers to SWE.5 by its pre-4.0 style name, “Software Integration & Integration Test” [12].

Traceability is a first-class, recurring construct in the model rather than an afterthought: “Ensure consistency and establish bidirectional traceability” is the **name** of a base practice in several processes, including SYS.2.BP5, SYS.3.BP4, SYS.4.BP4, SYS.5.BP4 and SWE.1.BP5 [12]. SWE.1 has seven process outcomes, of which outcomes 5 and 6 state that consistency and bidirectional traceability are established between software requirements and the system requirements, and between software requirements and the system architecture; SWE.1.BP5 accordingly requires consistency and bidirectional traceability between software requirements and **both** of those two targets [12]. The PAM is unusually candid about what this buys. Note 11 to SWE.1.BP5 reads: “Bidirectional traceability supports consistency, and facilitates impact analysis of change requests, and demonstration of verification coverage. Traceability alone, e.g., the existence of links, does not necessarily mean that the information is consistent with each other.” Note 9 adds: “Redundant traceability is not intended.” [12] Traceability also reaches down to code: SWE.3’s outcome 3 requires bidirectional traceability between software detailed design and software architecture, **and** between source code and software detailed design [12]. SWE.4 and SWE.6 extend it to verification measures and verification results [12]. Finally, “13-51 Consistency Evidence” is listed among SWE.1’s output information items [12] — traceability materialised as a work product.

The comparison is now sharp enough to state precisely. Automotive SPICE asks for traceability and consistency between work products, and then assesses whether an organisation’s **process** is capable of producing them, on a capability scale. speclink does something narrower and different in kind: it makes a subset of those links a **build precondition** in one specific technology stack. Table 5 lists the correspondences.

The omissions are larger than the correspondences and should be read first. speclink addresses nothing in requirements elicitation: it begins at a Markdown file or mockup that somebody already wrote, and the step from a source segment into a requirement has no formal semantics at all. It says nothing about stakeholder requirements,

nothing about system-level processes, and nothing about hardware. It does not touch the management or supporting process groups — project management, quality assurance, configuration management as a discipline, supplier monitoring — and it has no notion whatsoever of the capability dimension: there is no speclink analogue of a process attribute, a rating, or a capability level, and a green build is not a level-anything statement. Within the software engineering group it covers a strict subset, and only for the languages a profile exists for.

One version question should be recorded honestly. Version numbers beyond 4.0 circulate in the assessor community, but the VDA QMC’s own page refers to version 4.0 as the current one and links the v4.0 PDF [58], and no later PAM document could be retrieved from a primary source. Everything stated above is taken from version 4.0, and nothing here should be read as a statement about any successor.

5.3 What a compiler cannot deliver

Process assessment rates organisational capability. ISO/IEC 33002:2015 “defines the minimum set of requirements for performing an assessment that will ensure assessment results are objective, consistent, repeatable, and representative of the assessed processes” [7]; ISO/IEC 33004:2015 sets the requirements for the reference, assessment and maturity models themselves [8]. The subject of that judgement is an organisation and its processes — not a repository.

speclink’s subject is a repository. A green run is a statement about one working tree at one commit: that every source segment became a requirement, that every construct carrying business meaning names one, that every normative requirement is named by some construct, and that some test claimed and was recorded as having demonstrated it. It says nothing about the competence of the people involved, nothing about planning, review discipline or supplier management, and nothing about whether the requirements are the right requirements. It cannot even judge the one step above it: whether a requirement faithfully renders the paragraph it came from is not decidable, and speclink only records that the paragraph was read at the wording it then had.

The tool’s own documentation is blunt about the limit that matters most: coverage says only that code was written for a requirement, never that the code does what the requirement asks (README §10b; quoted in full in Section 8). That is why the **verified** direction exists at all, and why the figure beside it is a recorded test run rather than a marker in the source — and it is also the boundary past which no compiler, speclink included, can go.

6 DELIMITATION FROM SPEC-DRIVEN DEVELOPMENT PRACTICE

A practitioner movement has grown around the idea that a written specification should drive the generation of code by a language model. It travels under the name **spec-driven development**, and it is documented almost entirely in vendor repositories, product documentation, marketing pages and self-published video. It is not documented in peer-reviewed literature. This section says so explicitly, and then says why the movement still has to be addressed: it is what a growing number of practitioners actually do, and a paper that proposes a different place for the specification owes its reader a precise statement of what is and what is not being claimed relative to that practice.

One methodological remark governs everything below. None of the sources cited in this section is peer-reviewed. Two are vendor documentation, one is vendor marketing, one is a self-published video. Consequently no statement here supports an absolute claim about the state of the art, and every statement is attributed to the source that

makes it. Where a capability is not mentioned by a source, that is recorded as **not documented**, never as **absent**.

6.1 Generator-side specification

GitHub publishes Spec Kit, an MIT-licensed toolkit whose own README states that spec-driven development makes specifications “executable”, “directly generating working implementations rather than just guiding them” [22]. According to that documentation, a project is taken through a fixed sequence of slash-commands — establishing project principles, then specify, plan, tasks, implement and converge — with `/speckit.analyze` described as cross-artifact consistency and coverage analysis run between the task breakdown and the implementation [22]. The same README describes regulatory traceability and V-model test traceability not as built-in behaviour but as something a preset or a community extension **could** add [22].

Amazon’s Kiro is documented by its vendor as being built and operated by a team within AWS [13]. Per that documentation its specs feature generates three Markdown artefacts for each feature or bug fix — `requirements.md`, `design.md` and `tasks.md` — capturing user stories and acceptance criteria, technical design, and a plan of discrete trackable tasks respectively, with an LLM-based “Analyze Requirements” step offered to catch inconsistencies, ambiguities and gaps before design [13]. Jama Software, a requirements-management vendor, markets a spec-driven-development capability in which engineers and AI engineering agents iterate in a shared context over MCP [33]; this is a marketing statement on a product page, and is reported here only to show that the label has been adopted on both sides of the tooling landscape.

The analytic point is one of position, and it is the whole of this section. All three systems place the specification **in front of** the generator. The specification is an input to code production: it is authored, it is analysed, and it is then consumed by an agent that emits code. speclink places the specification **behind** the compiler. A `spec.Requirement` value and the `spec.Satisfies` binding that names it are ordinary Go terms in the ordinary build, and `speclink.verify` is a build step that runs after the code exists and exits non-zero on any finding; the four directions defined in Section 4 must all reach a hundred per cent for a run to be clean (README §2). It is indifferent to who or what wrote the code.

The two arrangements are complementary rather than competing, and a project could reasonably use both: a generator-side workflow to get from an idea to a first implementation, and a build-side check to keep the correspondence true afterwards. What separates them is duty over time. A specification consumed by a generator has discharged its role once the code has been emitted; structurally it has no further obligation, and nothing in the build notices if the code and the document diverge on the next commit. A specification that is a precondition of the build has an obligation at every subsequent commit, because the check is re-run and the failure is a red build rather than a stale document.

A second difference concerns who is trusted. Where an artefact is produced by a generator and then judged by the same generator, the judgement carries no independent weight, for the self-preference reason set out in Section 1 [49]. This is the reason speclink takes authorship and review records from outside the code entirely: `speclink attest -origin llm` and `speclink attest -reviewer` are recorded by the caller, never declared in the source, precisely because the same machine that writes the code would otherwise write the claim about who wrote it (README §2). speclink is explicit that this only shifts the question to who is permitted to make those calls, which is not speclink’s to decide.

Dimension	Generator-side spec tooling	Requirements-management tooling	speclink
Where the specification lives	Markdown artefacts in the repository [13, 22]	Items in the vendor’s own repository [33]	Go values in <code><ID>.spec.go</code> and bindings beside the code
When it is checked	During the agent workflow, before implementation [13, 22]	Not documented	After the Go build, as a build step
What happens on violation	Not documented	Not documented	Non-zero exit code; every finding is an error
What is checked	Cross-artifact consistency and coverage, by the agent [22]; requirements analysed for gaps and ambiguities [13]	Links between items; compliance and audit trails [33]	Source segments accounted, constructs bound, requirements covered and verified
Who is trusted	The generating agent	The tool’s users and its audit trail [33]	The compiler, plus records taken from outside the code
Coupling to language or stack	Language-independent Markdown [13, 22]	Language-independent; source control integrated externally [33]	Bound to the language: a profile per language, framework and style
Legacy applicability	Not documented	Not documented	Package by package, via <code>scope</code>

Table 6: Positioning of speclink against generator-side spec tooling and requirements-management tooling. Every cell attributed to a vendor source reports that vendor’s own documentation or marketing.

It should be stated plainly what speclink does not do. It does not generate code. It does not prompt a model. It takes no position on how the code came to exist, and offers no workflow, template or command for producing it.

6.2 A self-published field report

One frequently circulated practitioner account requires careful handling. The video *Der Moment, der die Softwareentwicklung geändert hat!* was published by David Tielke on his own YouTube channel and uploaded on 21 June 2026 [57]. It is not peer-reviewed, it names no venue or event, and it has undergone no independent review. Only the video description and its chapter list could be retrieved; the transcript was not retrievable, and the summary that follows is therefore derived solely from the description text.

From that description: the author reports a self-conducted, uncontrolled experiment in which an application he had used for eighteen years was rebuilt as an enterprise system — described as having a microservice architecture, local AI, a workflow engine, a full specification, tests and documentation — largely with AI agents. Per its own chapter list the work is staged through micro-management, quality-driven and spec-driven phases, then test-driven development from requirements, and finally idea- and voice-driven development, closing with self-reported figures on scope, code quality, cost and speed [57].

The methodological point follows without condescension. Self-reported figures from an uncontrolled single-subject experiment cannot establish anything, and this paper draws no quantitative claim from the report. What the account does legitimately illustrate is that the practitioner community has converged on specification-first workflows for agentic development — the same convergence visible in the vendor documentation above. That convergence is what motivates the question this paper addresses. Given such a workflow, what remains checkable by a machine? The answer speclink offers is: the trace, the structure and the evidence — that a requirement exists and is reachable from a source segment, that a construct names one, that a test claimed one and ran. Not the intent. Whether the code does the right thing remains a human judgement, which is why the review record is kept and why it is kept outside the code.

Honesty requires the symmetric observation. This paper presents no controlled experiment either — indeed no empirical evaluation at all, as Section 8 states without qualification. It reports a design and an implementation built and used inside one organisation by the people who wrote the tool. The criticism of the video’s method therefore

applies, in modified form, to this artefact as well, and no comparative claim about productivity, quality or cost is made anywhere in this paper.

6.3 Summary of positioning

Table 6 contrasts the three arrangements along the dimensions that distinguish them. Cells read **not documented** wherever the fetched sources are silent, which is the only supportable phrasing for an absence in a vendor document.

7 IMPLEMENTATION AND STATUS

speclink is a single Go binary with no runtime dependencies. PlantUML is a prerequisite of the environment rather than a dependency of the program: `speclink diagrams` writes `.puml` sources and runs nothing, so a checkout with no Java installed still executes every rule and every test.

7.1 Internal decomposition

The tree separates **reading a project, representing it, judging it and writing it out**.

Language front ends sit under `internal/lang`, behind a common interface. `internal/lang/golang` reads Go source through the type checker and holds the recognisers for both Go profiles: `kinds.go`, `infer.go` and `framework.go` decide what counts as a construct, `nago.go` and `bare.go` carry the two frameworks, `schema.go` reads persisted shapes, `endpoint.go` and `endpoint_hapi.go` read mounted routes, `trace.go` follows a route to the use case behind it, and `arch_context.go`, `arch_layout.go`, `arch_usecase.go`, `layering.go` and `rules.go` implement the style rules (the K4 to K8 families, which are language- and framework-specific and therefore live here rather than in `internal/check`). `internal/lang/jvm` reads a JVM project from compiled classes instead: `classfile/` is a hand-written class-file and constant-pool reader, `spring.go` recognises Spring’s annotations, and `verified.go` joins test claims to Surefire’s report.

`internal/ir` is the language-neutral intermediate representation both front ends produce: `construct.go`, `requirement.go`, `process.go`, `topology.go`, `endpoint.go`, `schema.go`, `entrypoint.go`, `packages.go`, `architecture.go`, `waivers.go`, and `dialect.go`, which supplies the wording a finding needs so that the rules can phrase a fix without knowing the language they are looking at.

`internal/check` holds the language-independent rules, one file per family: `structure.go` (K1-CONSTRUCT-UNBOUND), `fields.go` (K1-FIELD-UNBOUND), `persistence.go` (K1-PERSISTENCE-UNJUSTIFIED), `coverage.go` (the K3 requirement-coverage rules), `draft.go` (K9-DRAFT-REDUNDANT), `evolution.go` (the ten K9 schema-evolution rules against the baseline), `drift.go` (K10-REQ-CHANGED, K13-SOURCE-DRIFT), `sources.go` (K12-SOURCE-UNCOVERED), `verification.go` (the K14 evidence rules), `lifecycle.go` (K15), `process.go` (the fourteen K16 process rules), `topology.go` and `contract.go` (K17), `provenance.go` (K18-REVIEW-STALE) and `endpoint.go` (the nine K20 endpoint and wire rules).

The remainder is supporting machinery: `internal/source` loads Markdown documents and mockups and segments them into addressable parts (`markdown.go`, `image.go`, `slug.go`, `standard.go`); `internal/reqtree` orders and lays out the requirement tree and checks themes (`topic.go`); `internal/baseline` reads and writes `speclink.lock`, the record of frozen shapes, recorded test runs, reviews and promised addresses; `internal/doc` renders the specification document, with `markdown.go` and `typst.go` as the two backends; `internal/render` emits PlantUML (`c4.go`, `packages.go`, `puml.go`); `internal/diag` is the finding type and its set; `internal/config` reads `speclink.json`; and `internal/profile` registers profiles and holds the init templates.

7.2 Command surface

The commands are `init`, `diagrams`, `verify`, `requirements`, `freeze`, `inventory`, `impact`, `attest`, `evidence` and `generate`. `verify` exits with 1 if there is any finding and 0 otherwise; there are no warnings and no severities, and `scope` is the only dial. The build order is fixed:

```
go build ./...
speclink verify ./...
go test -json ./... | speclink evidence
```

`speclink inventory` reports what was recognised, which is the question `verify` cannot answer because a correctly recognised construct produces no output. `speclink impact` reports the reach of a change from a requirement, a document anchor or a path. `speclink attest` records authorship and review from outside the code. `speclink freeze` writes the baseline.

7.3 Profiles and configuration

A project names one profile in `speclink.json`: `go_nago_ddd1`, `go_bare_ddd1` or `java_springboot_ddd1`. The name has three parts because there are three decisions — language, framework, style — and `speclink` refuses to guess the third. `speclink.json` states deviations from the profile and never the conventions themselves; every profile understands `sourceRoots`, `scope` and `exclude`, and profile-specific keys beyond those are refused rather than ignored. `speclink init -describe` lists profiles, templates and their parameters, in text or JSON, because the usual caller is an agent for which an interactive prompt is the worst possible interface.

Two styles share one language and one reader. `go_bare_ddd1` adds four K6 rules that `go_nago_ddd1` has no use for, and two terms — `spec.Persistence()` and `spec.StoredAs[Domain]()` — which are findings under `go_nago_ddd1`, where the repository constructor already says the same thing.

7.4 Generated artefacts

`speclink generate` writes the specification document as Markdown or Typst; `internal/doc/typst.go` implements the latter.

This paper is typeset with Typst, the same output format the tool emits.

7.5 Measured facts

Go lines, tool source	24,433
Go lines, tests (*_test.go)	10,234
Go lines, testdata/ fixtures	3,222
func Test functions	349
Go packages (go list ./...)	16
Distinct rule IDs in README §11	77
Git commits	98
First commit	2026-08-10
Last commit	2026-08-30
go build ./...	passes
go test ./...	passes

Table 7: Measured on a clean checkout of commit 030ea34, dated 2026-08-30. Rule IDs are the distinct K... identifiers in the README rule index; three further identifiers exist in the code but not in that index (see below). Both `go build ./...` and `go test ./...` pass on a clean checkout of 030ea34.

The history is three weeks long, which is the plainest caveat available about the artefact’s maturity: the rules, the intermediate representation and the lock-file format have not yet been exposed to the kind of long-running project whose evolution they exist to constrain.

7.6 Discrepancies between the documentation and the code

Four are worth recording, because they bear on claims the tool makes about itself.

The Typst backend is undocumented as implemented. README §12 lists “any documentation backend other than the Markdown one” among the things that do not exist, but `internal/doc/typst.go` implements one, README §2 documents -format `markdown|typst`, and `cmd/speclink/typst_test.go` exercises it. README §12 is stale on this point.

Three rule identifiers exist in the code but not in the README index. K1-FIELD-UNBOUND (`internal/check/fields.go`), K1-PERSISTENCE-UNJUSTIFIED (`internal/check/persistence.go`) and K6-CTX-PRESENTATION-NO-IMPORT (`internal/lang/golang/describe.go`) can all be emitted, and the index a project would consult in order to waive one does not list them.

Six V6 numeric codes are assigned twice. SPEC-V6-090 through SPEC-V6-095 each name two different rules in the index. README §11 opens by stating that every rule ID is stable and may be used in `spec.Waive`; the K... names are unique, but the numeric codes that accompany them are not, and a reader filtering a JSON diagnostic stream on code would conflate two families.

The rule families are not all in internal/check. K4 to K8 are implemented in `internal/lang/golang` (`rules.go`, `layering.go`, `arch_context.go`, `arch_layout.go`, `arch_usecase.go`, `nago.go`) because they are style rules over a specific framework, and the two requirement-tree families K11 and K19 are in `internal/reqtree/topic.go`. The separation between language-independent and style-specific rules is real, but it is a property of the tree rather than of any stated interface.

7.7 What is not implemented

The following do not exist and must not be assumed:

- `speclink selfreport` and `verify --check-generated`;
- documentation backends other than Markdown and Typst — no HTML, no AsciiDoc, no JSON-LD;
- the K9 evolution rules for storage other than the JSON repository and the event log; a type written through some other store is not part of the promised set;
- any rule checking that a projection is not persisted, or that a repository is not reached from `ui*` beyond the existing import ban.

`java_springboot_ddd1` prescribes no style rules and has no persistence recogniser; a run under it says so rather than reporting those families sound.

Two blockers are recorded as measured rather than suspected.

Constructor naming: in the reference ERP, 193 use-case constructors are a naming convention, 45 a file convention and 9 genuine duplicates, and K5-UC-CONSTRUCTOR reports all of them alike. **Event identity:** whether the stable identifier of an event is its discriminator or a versioned type field is unsettled, and must be decided before the first context is frozen.

7.8 Availability

The repository is at github.com/worldiety/speclink. The LICENSE file is not an open-source licence: it states that use requires a licence agreement with worldiety GmbH.

8 DISCUSSION AND LIMITATIONS

The preceding sections argue that a trace link can be made a build precondition. This section states what that buys and what it does not.

8.1 What a green run means

A green `speclink verify` is a conjunction of the four directions (Section 4), asserted about one working tree at one commit. Every segment of every source document either yielded a requirement or was marked **informative**. Every recognised construct that must name a requirement names one, and the Go compiler resolved that name. Every **Normative** requirement is named by at least one construct, and no **Abstract** or **Superseded** requirement is. And for every normative requirement there exists a `spec.Verified` call, with `speclink evidence` reporting separately how many of those calls were observed to run to completion inside a passing test.

Each statement is about the **existence and shape of a link**, and none is about the behaviour of the program. The README states the boundary in one sentence: coverage says code was written for a requirement, and “It has never said the code does what the requirement asks” (README §10b). Evidence narrows the gap by one step and no more. A recorded run means that a test containing `spec.Verified(t, R)` reached that call and then passed. It does not mean the test asserted anything relevant to R, that its assertions were strong, or that R was the requirement the stakeholder needed. A test whose body is empty but for the marker produces the same evidence record as a thorough one; `speclink` has no notion of test adequacy and implements no coverage measure over the code under test.

The guarantee is therefore: **the artefacts are connected, the connections type-check, the wordings have not moved since somebody looked at them, and a passing test claimed each requirement**. Nothing weaker, and — this is the half that a design paper is tempted to slide past — nothing stronger. The verified/demonstrated pair of Section 4 reports an uncertainty about **execution**, not about **adequacy**; there is no further figure, and the design offers no path to one.

8.2 The unchecked link at the top

The step from a human-written source segment to a `spec.Requirement` has no formal semantics and cannot be given one, because it is a translation between a natural-language artefact and a typed value, and no property of the pair is decidable. Everything below it is conditional on it. If a requirement misstates the paragraph it was extracted from, then the compiler-checked binding, the coverage figure and the evidence record are all sound statements about a false premise, and they will all be green.

`speclink` offers four mitigations, and none of them is a check on the extraction itself. Segment coverage (K12-SOURCE-UNCOVERED) forces every part of every document to be **accounted for**, so nothing is silently skipped — but a segment is accounted for by any requirement, including a wrong one. The **informative** marker moves the decision that a segment carries no obligation into the document, next to the person who wrote it — but nobody checks that the marker is deserved. Fingerprints (K13-SOURCE-DRIFT, K10-REQ-CHANGED) establish that the segment still reads as it did when it was last frozen — which detects **change**, never **misreading**; a requirement wrong from the first commit is frozen wrong and stays green forever. Review recorded by `speclink freeze -reviewer` records that a named person saw a specific requirement wording (Section 4) — a record of an event, not a judgement about its quality, and `speclink` checks neither who made the call nor whether they read anything. Mitigation is not verification, and the four together do not amount to partial verification of a step that admits none.

The sharpest failure mode is closure. If one model both extracts the requirements from the source documents and writes the code that satisfies them, then the requirement tree and the implementation share an author and an interpretation. A misreading of the source document propagates coherently into both sides of a link that `speclink` then certifies as intact. The result is a system that is internally self-consistent, wrong with respect to what the stakeholder wrote, and verifies perfectly. Routing the check to a second model is not an escape, for the self-preference reason given in Section 1 [49]. `speclink`’s answer is to keep the review record outside the code and to print the pairing of requirement and segment for a person to read. That is an invitation to look, not a control.

8.3 The binding is a judgement

`spec.Satisfies(R)` is checked in exactly one respect: that R exists and is a requirement. That is the Go compiler’s work, and it is all the compiler can do. Whether **this** construct is the one that satisfies **this** requirement is a human or machine judgement expressed as a well-typed term, and a wrong binding is indistinguishable from a right one at every stage of the pipeline. Binding an arbitrary use case to an arbitrary normative requirement removes a K3-REQ-UNCOVERED finding and turns the build green. The mechanism that prevents a trace link from decaying into a stale string does nothing whatsoever to establish that the link was apt when it was written.

8.4 Structural coupling

`speclink` enforces an architecture as much as it records a trace. It is bound to a project layout (`<ID>.spec.go`, `<base>.annotation.go` beside `<base>.go`, `requirements/_sources`), to a profile naming a language, a framework and a style, and to a closed set of recognised constructs. The K4-K8 families are framework-specific by construction and live in the Go front end. A codebase that does not organise itself this way is not partially supported; it is unrecognised, and its constructs simply do not appear.

The cost is generality, and the traceability literature is where that cost is felt. Link recovery, reviewed in Section 2, exists precisely because most systems have no links [14, 29], and its own evaluations

report that it does not recover them completely [38]. `speclink` does not address that problem at all. Its `scope` key narrows adoption package by package, but the destination is still the layout. The answer `speclink` offers is available only to projects willing to be built its way — a genuine restriction on the class of systems the approach applies to, and one that excludes most existing ones.

8.5 Waivers and the incentive to stay silent

The waiver mechanism of Section 3 turns on a mandatory free-text reason, and mandatory free text is a weak control. `speclink` does not check, and cannot check, whether a reason is good; any non-empty string satisfies it. In a project where a machine is asked to make a build green, a waiver is the cheapest available move, and nothing in the design makes it expensive. The counter- pressures are structural rather than semantic: a waiver appears in the generated report and counts toward the figures, so it is visible in review; there are no severities, so there is no quieter place to move a finding to; and a waiver attaches to a Go construct, so it sits in the diff next to the code it excuses. Whether these pressures actually suppress abuse is unmeasured. No waiver counts from any project are reported in this paper, and no study of reviewer behaviour under this mechanism has been carried out.

8.6 Absence of empirical evaluation

This is the central limitation and it is unqualified. There is no controlled study, no comparison against a baseline, no multi-project dataset, and no measurement of anything. It is not shown that links maintained this way are more accurate than links kept in a requirements-management tool, nor cheaper to maintain, nor that they are maintained at all beyond the projects the authors control. It is not shown that a build-time failure produces a repair rather than a waiver. It is not shown that developers can read the diagnostics.

What such an evaluation would look like is known. The controlled experiment of Patrick Mäder and Alexander Egyed. [39] cited in Section 1 — subjects, tasks, a control condition, an effect measured — is a fair reference point for what is missing here. Patrick Rempel and Patrick Mäder. [53] is a second: 24 open-source projects and a multi-level regression relating traceability completeness to defect rate. Nothing of either kind was attempted.

The experience behind this paper is single-subject and non-independent. The tool has been used inside one organisation, on that organisation's own codebase, by the people who wrote the tool. Section 6 criticises a self-published practitioner field report for being an uncontrolled single-subject experiment with self-reported figures [57]. That criticism applies to this artefact in the same form and with the same force, and the symmetry is not rhetorical politeness: every figure in Section 7, including the constructor-naming counts from the reference ERP, was measured by the tool's authors on their own project with their own tool, and carries exactly the evidential weight that arrangement allows.

8.7 Tool trust and self-application

`speclink` produces evidence about other software while being unqualified software itself. Section 5 says so; it bears repeating, because the claim/evidence distinction the design applies to its subjects applies to it. A finding is a claim by `speclink`, and nothing external corroborates it. `go vet`'s own documentation is explicit that its checks are heuristic guidance rather than a guarantee of correctness [23]; `speclink`'s rules are in the same category and have not been reviewed by anyone outside the project.

Self-application is worse than partial. The repository contains no `speclink.json`, no `requirements/` tree and no `speclink.lock`; the only `.spec.go` and `.annotation.go` files

in it are test fixtures under `testdata/`. `speclink` is therefore not governed by `speclink` at any level. The tool's rules about requirement coverage, evidence and review provenance have never been applied to the codebase that implements them, and no `speclink selfreport` command exists in `cmd/speclink`, so the capability to do so does not exist — README §12 also lists it as absent, though that section is stale on another point (Section 7). A tool that does not consume its own output is making a claim about maintainability under its regime that it has not tested on the one project it fully controls.

8.8 Threats to the validity of this paper's claims

Four, briefly. First, the comparison to functional-safety and aviation standards in Section 5 rests on publicly visible metadata — titles, scope lists, catalogue descriptions and Online Browsing Platform definitions — because the normative text is paywalled. No clause content was read, and a structural affinity argued from a table of contents is a weak argument. Automotive SPICE is the sole exception, its PAM being freely published [12]. Second, every statement about the requirements-management tool landscape derives from vendor documentation and marketing pages, is attributed as such, and can report only that a mechanism is **not documented** — never that it is absent. Third, the artefact has a three-week commit history (Section 7); no rule, no intermediate representation and no lock-file format in it has survived contact with a long-running project, which is precisely the setting the design exists for. Fourth, this paper was largely produced by a language model under human direction; Section 10 sets out what was done by a machine, what was done by a person and what controls were applied. That arrangement places the paper in the same category as the artefact it describes — generated text, sampled review.

9 CONCLUSION AND FUTURE WORK

The distinction between a claim and a piece of evidence is the core of this paper, and everything else in it is an application of that distinction. A claim is something the source says: a binding stating that a construct satisfies a requirement, a marker stating that a test verifies one, a name stating that somebody reviewed something. A claim is readable statically, which is exactly why a **missing** claim can be reported, and exactly why a **present** claim proves nothing: a marker can sit behind a condition that never holds, and a self-declared record of human authorship is not evidence about the author. Evidence is a record that something happened — a test that ran to the marker and passed, an attestation made from outside the code — and can never be re-derived from a working tree. `speclink` keeps the two apart everywhere, reports them as separate figures, and treats the gap between them as the number that carries information.

What was built is an annotation compiler with a Go front end and a JVM bytecode front end over a shared, language-neutral requirement model. It annotates only the one fact no analysis can infer, which requirement a construct exists for, and makes the annotation of any inferable fact a finding; every finding is an error and the only escape hatch is a waiver carrying a written reason. The annotation grammar is a closed whitelist inside the ordinary build, so references to requirements are resolved by the host language's compiler rather than by a parser of `speclink`'s own. Requirements are anchored to segments of Markdown documents and to named regions of mockups; their wordings and those segments are fingerprinted in `speclink.lock`, so a change is a diff in a pull request rather than a silent divergence. The chain ends at recorded passing test runs supplied by the build, and authorship and review are recorded by commands rather than declared in the source. What was compared is narrower than it may appear: a structural comparison against the publicly verifiable parts of ISO 26262, IEC 61508, DO-178C and their neighbours, a compar-

ison against Automotive SPICE that could be made against primary normative text because the model is freely published, and a delimitation from generator-side spec-driven tooling and from requirements-management platforms. None of these is a conformance claim, a qualification argument, or a statement about a capability level.

The most consequential piece of future work is the one the paper does not contain: a controlled empirical evaluation with an independent baseline — subjects, tasks, a control condition and a measured effect, in the shape of Patrick Mäder and Alexander Egyed. [39], or a multi-project regression in the shape of Patrick Rempel and Patrick Mäder. [53] — conducted by people who did not write the tool, on projects the tool’s authors do not control. Second, speclink does not govern itself: the repository contains no requirement tree, no configuration and no lock file, and the command that would produce a self-report does not exist. Building it and applying the regime to the implementation is the cheapest available test of whether the discipline survives a long-running project. Third, the approach is bound to an enforced layout, a fixed set of recognised constructs and a profile naming a language, a framework and a style; broadening it — or characterising precisely which projects it excludes — is open. Fourth, whether the counter-pressures on waivers actually suppress abuse is unmeasured; waiver counts, and reviewer behaviour when a build fails, can be measured and have not been. Fifth, the JVM front end prescribes no style rules and reads no persisted shapes, and bringing a second language front end to parity would show what in the design is genuinely language-independent and what merely has not been ported. Sixth, and hardest: the step from a source segment to a requirement has no formal semantics, and it is unclear whether any assessable quality measure can be attached to it at all. Fingerprints detect that a segment moved; nothing detects that it was misread. Until that question has an answer, every figure below it is a sound statement about a premise nobody checked.

10 DISCLOSURE OF AI INVOLVEMENT

This paper reports work in which a large language model was not an assistant at the margins but the executing agent for most of the work described. The disclosure below is deliberately specific, because a generic acknowledgement would tell a reader nothing they could act on, and because the paper’s own argument is that a claim made by the party with an interest in it is not evidence.

10.1 What was done by a machine

The following were produced by a large language model operating under the direction of the named author:

Research the literature and standards survey. Every source in the bibliography was located, retrieved and its metadata extracted by an agent. No source was written down from the model’s parametric memory.

Implementation substantially all of the source code of speclink, including the front ends, the rule implementations, the intermediate representation and the command-line surface.

Tests substantially all of the test suite and the fixtures under `testdata/`, including the worked example.

Text the wording of this paper. Each technical section was drafted by a separate agent instance from the repository and the verified research notes, and the sections were then assembled, deduplicated and made consistent by further agents.

Measurement the figures reported in the implementation section were produced by an agent running commands against a clean checkout of the repository.

10.2 What was done by a person

The named author defined the problem, the design rules that the tool enforces and the argument the paper makes; selected and rejected approaches; directed each agent; and reviewed and accepted the result. The author is responsible for every claim in this paper, including those he did not personally type.

10.3 The controls that were applied

Because the risk in this arrangement is fabrication rather than error, the process was structured against it rather than trusting the output:

1. Research and authoring were separated. Section authors were forbidden to perform their own research and could cite only from a bibliography prepared in advance.
2. Every source was required to be accompanied by a URL retrieved during the session and a verbatim quotation from the retrieved page. Anything that could not be retrieved was recorded as unverified and was excluded from the bibliography rather than silently softened.
3. An independent audit agent, which had not gathered the sources, re-fetched every one of them, cross-checked bibliographic metadata against at least two independent services where available, and string-matched every quotation against the retrieved page. It found no fabricated source and seventeen metadata or quotation defects, all of which were corrected before drafting began. Its report is retained in the repository alongside the research notes.
4. Section authors were bound by a written contract stating, per source, what that source may and may not be used to support. Paywalled standards may support statements about their titles, scopes, published definitions and publicly listed clause headings, and nothing else. Vendor documentation, marketing material and self-published video may support only attributed statements, never absolute ones.
5. A final review pass, performed by an agent that did not write any section, checked the assembled paper for internal contradiction and for claims exceeding their sources.

10.4 What these controls do not establish

They reduce the probability of an invented citation; they do not eliminate it, and a reader who intends to rely on a claim in this paper should check its source. They say nothing about whether the design is sound, whether the implementation is correct beyond what its own test suite demonstrates, or whether the argument is the right one. The paper’s central position — that a claim made by the author of an artefact about that artefact is not evidence about it — applies to this section as much as to any annotation in a source file. The retained research notes, the audit report and the repository history are offered so that the claim can be checked rather than believed.

11 AVAILABILITY

The tool, the worked example, the research notes gathered for this paper, the independent source audit and the sources of this document are in the repository at github.com/worldiety/speclink. The paper’s sources are under `doc/paper/`.

REFERENCES

- [1] 2010. IEC 61508-1:2010 — Functional safety of electrical/electronic/programmable electronic safety-related systems — Part 1: General requirements. Retrieved from <https://webstore.iec.ch/en/publication/5515>
- [2] 2010. IEC 61508-3:2010 — Functional safety of electrical/electronic/programmable electronic safety-related systems — Part 3: Software requirements. Retrieved from <https://webstore.iec.ch/en/publication/5517>

- [3] 2011. DO-178C — Software Considerations in Airborne Systems and Equipment Certification. Retrieved from <https://my.rtca.org/productdetails?id=a1B36000001lcmqEAC>
- [4] 2011. DO-330 — Software Tool Qualification Considerations. Retrieved from <https://my.rtca.org/productdetails?id=a1B36000001lcfIEAC>
- [5] 2011. DO-333 — Formal Methods Supplement to DO-178C and DO-278A. Retrieved from <https://my.rtca.org/productdetails?id=a1B36000001lcfFEAC>
- [6] 2015. IEC 62304:2006+AMD1:2015 CSV — Medical device software — Software life cycle processes. Retrieved from <https://webstore.iec.ch/en/publication/22794>
- [7] 2015. ISO/IEC 33002:2015 — Information technology — Process assessment — Requirements for performing process assessment. Retrieved from <https://www.iso.org/standard/54176.html>
- [8] 2015. ISO/IEC 33004:2015 — Information technology — Process assessment — Requirements for process reference, process assessment and maturity models. Retrieved from <https://www.iso.org/standard/54178.html>
- [9] 2018. ISO 26262-8:2018 — Road vehicles — Functional safety — Part 8: Supporting processes. Retrieved from <https://www.iso.org/standard/68390.html>
- [10] 2018. ISO 26262-6:2018 — Road vehicles — Functional safety — Part 6: Product development at the software level. Retrieved from <https://www.iso.org/standard/68388.html>
- [11] 2018. ISO 26262-1:2018 — Road vehicles — Functional safety — Part 1: Vocabulary. Retrieved from <https://www.iso.org/standard/68383.html>
- [12] 2023. Automotive SPICE® Process Reference Model / Process Assessment Model, Version 4.0. Retrieved from <https://vdqm.de/wp-content/uploads/2023/12/Automotive-SPICE-PAM-v40.pdf>
- [13] Amazon Web Services. Kiro Documentation — Specs. Retrieved from <https://kiro.dev/docs/specs/>
- [14] Giuliano Antoniol, Gerardo Canfora, Gerardo Casazza, Andrea De Lucia, and Ettore Merlo. 2002. Recovering traceability links between code and documentation. *IEEE Transactions on Software Engineering* 28, 10 (2002), 970–983. <https://doi.org/10.1109/TSE.2002.1041053>
- [15] Giuliano Antoniol, Gerardo Canfora, Gerardo Casazza, Andrea De Lucia, and Ettore Merlo. 2025. Recovering Traceability Links Between Code and Documentation: A Retrospective. *IEEE Transactions on Software Engineering* 51, 3 (2025), 825–832. <https://doi.org/10.1109/TSE.2025.3534027>
- [16] ArchUnit Project. ArchUnit — Unit test your Java architecture. Retrieved from <https://www.archunit.org/>
- [17] Lilian Burdy, Yoonsik Cheon, David R. Cok, Michael D. Ernst, Joseph R. Kiniry, Gary T. Leavens, K. Rustan M. Leino, and Erik Poll. 2003. An overview of JML tools and applications. *Electronic Notes in Theoretical Computer Science* 80, (2003), 75–91. [https://doi.org/10.1016/S1571-0661\(04\)80810-7](https://doi.org/10.1016/S1571-0661(04)80810-7)
- [18] Lilian Burdy, Yoonsik Cheon, David R. Cok, Michael D. Ernst, Joseph R. Kiniry, Gary T. Leavens, K. Rustan M. Leino, and Erik Poll. 2005. An overview of JML tools and applications. *International Journal on Software Tools for Technology Transfer* 7, 3 (2005), 212–232. <https://doi.org/10.1007/s10009-004-0167-4>
- [19] Jane Cleland-Huang, Orlena Gotel, and Andrea Zisman (Eds.). 2012. *Software and Systems Traceability*. Springer, London. <https://doi.org/10.1007/978-1-4471-2239-5>
- [20] dominikh/go-tools maintainers. Staticcheck Documentation. Retrieved from <https://staticcheck.dev/docs/>
- [21] EUROCAE. ED-12C — Software considerations in airborne systems and equipment certification (document listing). Retrieved from <https://www.eurocae.net/?s=ED-12C>
- [22] GitHub, Inc. Spec Kit — Toolkit to help you get started with Spec-Driven Development. Retrieved from <https://github.com/github/spec-kit>
- [23] The Go Authors. Command vet. Retrieved from <https://pkg.go.dev/cmd/vet>
- [24] The Go Authors. Package golang.org/x/tools/go/analysis. Retrieved from <https://pkg.go.dev/golang.org/x/tools/go/analysis>
- [25] Orlena C. Z. Gotel and Anthony Finkelstein. 1994. An analysis of the requirements traceability problem. In *Proceedings of the IEEE International Conference on Requirements Engineering (ICRE)*, 1994. 94–101. <https://doi.org/10.1109/ICRE.1994.292398>
- [26] Orlena Gotel, Jane Cleland-Huang, Jane Huffman Hayes, Andrea Zisman, Alexander Egyed, Paul Grünbacher, and Giuliano Antoniol. 2012. The quest for Ubiquity: A roadmap for software and systems traceability research. In *Proceedings of the 20th IEEE International Requirements Engineering Conference (RE)*, 2012. 71–80. <https://doi.org/10.1109/RE.2012.6345841>
- [27] Orlena Gotel, Jane Cleland-Huang, Jane Huffman Hayes, Andrea Zisman, Alexander Egyed, Paul Grünbacher, Alex Dekhtyar, Giuliano Antoniol, and Jonathan I. Maletic. 2012. The Grand Challenge of Traceability (v1.0). In *Software and Systems Traceability*. Springer, London, 343–409. https://doi.org/10.1007/978-1-4471-2239-5_16
- [28] Orlena Gotel, Jane Cleland-Huang, Jane Huffman Hayes, Andrea Zisman, Alexander Egyed, Paul Grünbacher, Alex Dekhtyar, Giuliano Antoniol, Jonathan I. Maletic, and Patrick Mäder. 2012. Traceability Fundamentals. In *Software and Systems Traceability*. Springer, London, 3–22. https://doi.org/10.1007/978-1-4471-2239-5_1
- [29] Jane Huffman Hayes, Alex Dekhtyar, and Senthil Karthikeyan Sundaram. 2006. Advancing candidate link generation for requirements tracing: the study of methods. *IEEE Transactions on Software Engineering* 32, 1 (2006), 4–19. <https://doi.org/10.1109/TSE.2006.3>
- [30] Sebastian Herold and Andreas Rausch. 2014. A Rule-Based Approach to Architecture Conformance Checking as a Quality Management Measure. In *Relating System Quality and Software Architecture*. 181–207. <https://doi.org/10.1016/B978-0-12-417009-4.00007-7>
- [31] IBM. IBM Engineering Requirements Management DOORS Family. Retrieved from <https://www.ibm.com/products/requirements-management>
- [32] Daniel Jackson. 2002. Alloy: a lightweight object modelling notation. *ACM Transactions on Software Engineering and Methodology* 11, 2 (2002), 256–290. <https://doi.org/10.1145/505145.505149>
- [33] Jama Software. Jama Connect. Retrieved from <https://www.jamasoftware.com/platform/jama-connect/>
- [34] Gary T. Leavens, Albert L. Baker, and Clyde Ruby. 1999. JML: A Notation for Detailed Design. In *Behavioral Specifications of Businesses and Systems*. Springer, Boston, MA, 175–188. https://doi.org/10.1007/978-1-4615-5229-1_12
- [35] Gary T. Leavens, Albert L. Baker, and Clyde Ruby. 2006. Preliminary design of JML: a behavioral interface specification language for Java. *ACM SIGSOFT Software Engineering Notes* 31, 3 (2006), 1–38. <https://doi.org/10.1145/1127878.1127884>
- [36] Eduardo F. de Lima and Ricardo Terra. 2020. ArchPython: architecture conformance checking for Python systems. In *Proceed-*

- ings of the XXXIV Brazilian Symposium on Software Engineering (SBES), 2020. 772–777. <https://doi.org/10.1145/3422392.3422505>
- [37] Zhiyong Liu, Huanchao Chen, Xiangping Chen, Xiaonan Luo, and Fan Zhou. 2018. Automatic Detection of Outdated Comments During Code Changes. In *Proceedings of the IEEE 42nd Annual Computer Software and Applications Conference (COMPSAC)*, 2018. 154–163. <https://doi.org/10.1109/COMPSAC.2018.00028>
- [38] Andrea De Lucia, Fausto Fasano, Rocco Oliveto, and Genoveffa Tortora. 2007. Recovering traceability links in software artifact management systems using information retrieval methods. *ACM Transactions on Software Engineering and Methodology* 16, 4 (2007), 13. <https://doi.org/10.1145/1276933.1276934>
- [39] Patrick Mäder and Alexander Egyed. 2012. Assessing the effect of requirements traceability for software maintenance. In *Proceedings of the 28th IEEE International Conference on Software Maintenance (ICSM)*, 2012. 171–180. <https://doi.org/10.1109/ICSM.2012.6405269>
- [40] Patrick Mäder and Alexander Egyed. 2015. Do developers benefit from requirements traceability when evolving and maintaining a software system?. *Empirical Software Engineering* 20, 2 (2015), 413–441. <https://doi.org/10.1007/s10664-014-9314-z>
- [41] Bruno Menezes, Ana Teresa C. Martins, and Thiago Alves Rocha. 2021. A Two-Level Approach Based on Model Checking to Support Architecture Conformance Checking. In *Formal Methods: Foundations and Applications (SBMF)*, 2021. Springer, 1–16. https://doi.org/10.1007/978-3-030-92137-8_1
- [42] Bertrand Meyer. 1992. Applying "Design by Contract". *Computer* 25, 10 (1992), 40–51. <https://doi.org/10.1109/2.161279>
- [43] Gail C. Murphy, David Notkin, and Kevin J. Sullivan. 1995. Software reflexion models: bridging the gap between source and high-level models. In *Proceedings of the 3rd ACM SIGSOFT Symposium on Foundations of Software Engineering (FSE)*, 1995. 18–28. <https://doi.org/10.1145/222124.222136>
- [44] Gail C. Murphy, David Notkin, and Kevin J. Sullivan. 2001. Software reflexion models: bridging the gap between design and implementation. *IEEE Transactions on Software Engineering* 27, 4 (2001), 364–380. <https://doi.org/10.1109/32.917525>
- [45] OASIS Open. Open Services for Lifecycle Collaboration (OSLC). Retrieved from <https://open-services.net/>
- [46] Object Management Group. Requirements Interchange Format™ (ReqIF™). Retrieved from <https://www.omg.org/reqif/>
- [47] Oracle. Interface Processor — package javax.annotation.processing, module java.compiler. Retrieved from <https://docs.oracle.com/en/java/javase/21/docs/api/java.compiler/javax/annotation/processing/Processor.html>
- [48] Ipek Ozkaya. 2023. Infrastructure as Code and Software Architecture Conformance Checking. *IEEE Software* (2023), 4–8. <https://doi.org/10.1109/MS.2022.3213880>
- [49] Arjun Panickssery, Samuel R. Bowman, and Shi Feng. 2024. LLM Evaluators Recognize and Favor Their Own Generations. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. <https://doi.org/10.48550/arXiv.2404.13076>
- [50] Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh. 2023. Do Users Write More Insecure Code with AI Assistants?. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2023. 2785–2799. <https://doi.org/10.1145/3576915.3623157>
- [51] PTC Inc. Codebeamer. Retrieved from <https://www.ptc.com/en/products/codebeamer>
- [52] Balasubramaniam Ramesh and Matthias Jarke. 2001. Toward reference models for requirements traceability. *IEEE Transactions on Software Engineering* 27, 1 (2001), 58–93. <https://doi.org/10.1109/32.895989>
- [53] Patrick Rempel and Patrick Mäder. 2017. Preventing Defects: The Impact of Requirements Traceability Completeness on Software Quality. *IEEE Transactions on Software Engineering* 43, 8 (2017), 777–797. <https://doi.org/10.1109/TSE.2016.2622264>
- [54] Siemens Digital Industries Software. Polarion Requirements. Retrieved from <https://www.siemens.com/en-us/products/polarion/requirements/>
- [55] Mahesh De Silva and Indika Perera. 2015. Preventing software architecture erosion through static architecture conformance checking. In *Proceedings of the 10th International Conference on Industrial and Information Systems (ICIIS)*, 2015. 43–48. <https://doi.org/10.1109/ICIINFS.2015.7398983>
- [56] Nataliia Stulova, Arianna Blasi, Alessandra Gorla, and Oscar Nierstrasz. 2020. Towards Detecting Inconsistent Comments in Java Source Code Automatically. In *Proceedings of the 20th IEEE International Working Conference on Source Code Analysis and Manipulation (SCAM)*, 2020. 65–69. <https://doi.org/10.1109/SCAM51674.2020.00012>
- [57] David Tielke. 2026. Der Moment, der die Softwareentwicklung geändert hat!. Retrieved from <https://www.youtube.com/watch?v=eLDHrqKpIvI>
- [58] VDA QMC. Automotive SPICE®. Retrieved from <https://vda-qmc.de/en/automotive-spice/>
- [59] Viper Project, ETH Zürich. Gobra — an automated, modular verifier for Go programs. Retrieved from <https://github.com/viperproject/gobra>